
APLICACIONES MOVILES I

Tutorial Básico de Programación en Java



11 DE JUNIO DE 2026
DANIEL ROMAN TERRAZAS
UNIVERSIDAD PRIVADA DOMINGO SAVIO

Contenido

Tutorial Básico de Programación en Java	2
1. Tipos de Datos Primitivos Comunes	2
2. Operadores en Java	3
2.1. Operadores Matemáticos (Aritméticos)	3
2.2. Operadores Lógicos	3
3. Estructuras Condicionales	4
3.1. if, else if, else	4
4. Arrays (Arreglos)	4
5. Estructuras de Control de Flujo (Bucles)	5
5.1. Bucle for	5
5.2. Bucle while	5
6. Interacción con el Usuario: System.out y System.in	6
6.1. Salida de Datos: System.out	6
6.2. Entrada de Datos: System.in y Scanner	7
Ejemplo de Programa Completo	8

Tutorial Básico de Programación en Java

Java es un popular **lenguaje de programación** de alto nivel, orientado a objetos y con una filosofía de “escribe una vez, ejecuta en cualquier lugar” (**WORA**), gracias a que su código se compila a un *bytecode* que puede ejecutarse en cualquier máquina que tenga instalada la **Máquina Virtual de Java (JVM)**.



PhD en informática

Fue desarrollado inicialmente por **James Gosling** y su equipo en **Sun Microsystems** (conocido como el “Green Team”) a principios de la década de 1990, con su primera versión pública lanzada en 1995. Gosling es a menudo referido como “el padre de Java”. Es ampliamente utilizado para desarrollar una vasta gama de aplicaciones, desde **sistemas empresariales** a gran escala y **aplicaciones web** hasta aplicaciones **móviles** y software para **dispositivos embebidos**. Su diseño enfatiza la **portabilidad**, la **robustez** y la **seguridad**.

1. Tipos de Datos Primitivos Comunes

Los tipos de datos definen el tipo de valor que puede contener una variable.

- **int (Entero):** Almacena números enteros (sin decimales). Es el tipo más común para contar o manejar cantidades. `Java int edad = 30; int numeroVidas = 9;`
- **boolean (Booleano):** Almacena solo dos posibles valores: `true` (verdadero) o `false` (falso). Se utiliza para lógica y

```
condiciones.Javaboolean esMayorDeEdad = true; boolean  
programaActivo = false;
```

- **String (Cadena de Texto):** A diferencia de int y boolean, String **no es un tipo primitivo**, sino una clase. Se utiliza para almacenar secuencias de caracteres (texto). Se declara entre **comillas dobles** `""`. `JavaString nombre = "Juan Pérez"; String saludo = "Hola Mundo";`

2. Operadores en Java

2.1. Operadores Matemáticos (Aritméticos)

Se utilizan para realizar cálculos matemáticos.

Operador	Nombre	Ejemplo	Resultado
+	Suma	5 + 3	8
-	Resta	10 - 4	6
*	Multiplicación	2 * 6	12
/	División	10 / 2	5
%	Módulo (Resto)	10 % 3	1

2.2. Operadores Lógicos

Se usan para combinar o modificar expresiones booleanas.

Operador	Nombre	Descripción
&&	AND Lógico	Devuelve true si ambas condiciones son true.
	OR Lógico	Devuelve true si ambas una condición es true.

Operador	Nombre	Descripción
!	NOT Lógico	Invierte el valor booleano (convierte true a false y viceversa).

Ejemplo de Uso:

```
int a = 5;
int b = 10;
boolean condicion1 = (a > 0) && (b > 0); // true && true -> true
boolean condicion2 = (a > 10) || (b == 10); // false || true -> true
boolean condicion3 = !condicion1;          // !true      -> false
```

3. Estructuras Condicionales

Permiten que el programa tome decisiones y ejecute diferentes bloques de código según si una condición es verdadera o falsa.

3.1. if, else if, else

```
int nota = 85;

if (nota >= 90) {
    System.out.println("Sobresaliente");
} else if (nota >= 70) {
    // Se ejecuta si 'nota' no fue >= 90, pero sí es >= 70
    System.out.println("Notable");
} else {
    // Se ejecuta si no se cumplió ninguna de las condiciones anteriores
    System.out.println("Insuficiente");
}
```

Java

4. Arrays (Arreglos)

Un **Array** es un contenedor que puede contener un número fijo de valores del mismo tipo (ej. solo ints, solo Strings). En Java, los arrays se indexan comenzando desde 0.

Declaración e Inicialización

```
// 1. Array de enteros
int[] numeros = {10, 20, 30, 40, 50};

// 2. Array de Strings
String[] nombres = new String[3]; // Crea un array de 3 posiciones
nombres[0] = "Ana";                // Asigna un valor a la posición 0
nombres[1] = "Luis";               // Asigna un valor a la posición 1
nombres[2] = "Marta";             // Asigna un valor a la posición 2

// Acceder a un elemento:
System.out.println(numeros[2]); // Imprime: 30
```

Java

5. Estructuras de Control de Flujo (Bucles)

Los bucles permiten ejecutar un bloque de código repetidamente.

5.1. Bucle for

Ideal para iterar un número **conocido** de veces, comúnmente usado para recorrer arrays.

Sintaxis: for (inicialización; condición; actualización)

```
// Ejemplo 1: Imprimir números del 0 al 4
for (int i = 0; i < 5; i++) {
    System.out.println("Contador: " + i);
}

// Ejemplo 2: Recorrer un array
String[] frutas = {"Manzana", "Pera", "Uva"};
for (int i = 0; i < frutas.length; i++) {
    System.out.println("Fruta en la posición " + i + ": " + frutas[i]);
}
```

Java

5.2. Bucle while

Ideal para iterar mientras una **condición** sea **verdadera**, cuando el número de repeticiones es **desconocido** de antemano.

Sintaxis: while (condición)

```
int contador = 0;
while (contador < 3) {
    System.out.println("Iteración " + contador);
    contador++; // ¡Importante! Asegurarse de que la condición cambie
}
// El código se detiene cuando 'contador' es 3
```

Java

6. Interacción con el Usuario: System.out y System.in

6.1. Salida de Datos: system.out

La clase **System.out** es un flujo de salida estándar que se utiliza para **mostrar datos en la consola** (pantalla).

- **System.out.println():** Imprime la información y luego salta a una nueva línea.
- **System.out.print():** Imprime la información y permanece en la misma línea.

Ejemplo:

```
public class SalidaDatos {
    public static void main(String[] args) {
        // Imprime y salta de línea
        System.out.println("Este mensaje estará");

        // Imprime en la misma línea
        System.out.print("junto a este otro.");

        // Imprime un valor de variable
        int valor = 5;
        System.out.println(" El valor es: " + valor);
    }
}
// Salida:
// Este mensaje estará
// junto a este otro. El valor es: 5
```

6.2. Entrada de Datos: `System.in` y `Scanner`

La clase `System.in` es un flujo de entrada estándar que se utiliza para **leer datos** introducidos por el teclado.

Directamente usar `System.in` es complejo, por lo que en Java se utiliza la clase `Scanner` para interpretar y procesar fácilmente la entrada de texto como diferentes tipos de datos (enteros, cadenas, booleanos, etc.).

Pasos para la Entrada de Datos:

1. **Importar `Scanner`:** Se debe importar la clase al inicio del archivo: `import java.util.Scanner;`
2. **Crear el objeto `Scanner`:** Inicializar un objeto de la clase `Scanner`, pasándole `System.in`.
3. **Usar métodos de lectura:** Utilizar los métodos del objeto `Scanner` para leer el tipo de dato deseado.

Método de Scanner	Lee el Siguiente...
<code>.nextInt()</code>	<code>int</code> (entero)
<code>.nextDouble()</code>	<code>double</code> (decimal)
<code>.next()</code>	<code>String</code> (una sola palabra)
<code>.nextLine()</code>	<code>String</code> (toda la línea hasta presionar Enter)

Ejemplo Práctico:

```
import java.util.Scanner; // Paso 1: Importar la clase

public class EntradaDatos {
    public static void main(String[] args) {
        // Paso 2: Crear el objeto Scanner usando System.in
        Scanner lector = new Scanner(System.in);

        // --- Leer un String (una palabra) ---
        System.out.print("Ingresa tu nombre: ");
```



```

        String nombreUsuario = lector.next();

        // --- Leer un int ---
        System.out.print("Ingresa tu edad: ");
        int edadUsuario = lector.nextInt();

        // --- Mostrar el resultado usando System.out ---
        System.out.println("\n¡Hola, " + nombreUsuario +
"! Tienes " + edadUsuario + " años.");

        // Cierra el objeto Scanner al finalizar para liberar recursos
        lector.close();
    }
}

```

Ejemplo de Programa Completo

```

public class TutorialJava {
    public static void main(String[] args) {
        // 1. Tipos de Datos y Variables
        int edad = 25;
        String mensaje = "Hola a todos";
        boolean tienePermiso = true;

        // 2. Operadores Matemáticos
        int resultado = 10 * 5 + (15 % 4); // 50 + 3 = 53

        // 3. Estructura Condicional
        if (edad >= 18 && tienePermiso) {
            System.out.println("Puede acceder.");
        } else {
            System.out.println("Acceso denegado.");
        }

        // 4. Array
        String[] nombres = {"Elena", "Carlos", "David"};

        // 5. Bucle for para recorrer el array
    }
}

```

```
System.out.println("\n--- Lista de Nombres ---");
for (int i = 0; i < nombres.length; i++) {
    System.out.println("Nombre #" + (i + 1) + ": " + nombres[i]);
}

// 6. Bucle while
int cuentaAtras = 3;
while (cuentaAtras > 0) {
    System.out.println("Quedan: " + cuentaAtras);
    cuentaAtras--;
}
}
```